

GemmAI: Direct-Action Design for Offline Coding Agents on Small Gemma Models

Lessons from replacing verbose ReAct with a file-block protocol, self-healing execution, and git checkpoints on CPU-only consumer hardware

Max Hounkpatin (Max_Adis), MAX_ADIS, Porto-Novo, Benin

ABSTRACT

Most coding agents assume a cloud-hosted frontier model, stable internet, and capable hardware, which many developers, including students in West Africa, lack. We present GemmAI, an offline, terminal-based developer agent that runs entirely on consumer hardware (16 GB RAM or less, CPU-only, no cloud API) using small open models served locally through Ollama, with hot switching between models via a `/model` command. GemmAI was exercised with Gemma 2B (`gemma2:2b`) and with Gemma 4 through Ollama (`gemma4`), with the E2B edge variant as the low-memory target. Our first prototype followed the verbose ReAct pattern (Thought / Action / Observation). With small models it failed in recurring ways: the model parroted few-shot examples from the prompt, entered exploration loops on empty directories, and produced tool calls whose nested quotes broke the parser. We describe a redesign toward a *direct-action* architecture: the model writes labeled Markdown file blocks, the harness injects project context automatically, a fallback contextual parser recovers malformed output, and an intent filter separates conversation from development requests. Safeguards move reliability from the model into the harness: a self-healing execute-and-repair loop, an automatic git checkpoint before every operation with `/undo`, colored unified diffs, and multi-step write verification. We contribute a qualitative failure-mode taxonomy, a harness design addressing each failure, and a proposed reproducible evaluation protocol. We report no benchmark numbers; evaluation is explicitly future work.

1. Introduction

Coding agents built on large language models can now navigate repositories, edit files, and run tests [Yang et al., 2024; Jimenez et al., 2023]. Most depend on a hosted frontier model, assuming always-on bandwidth, per-token billing, and willingness to send source code to a third party. For many developers these assumptions do not hold. In Benin and in much of Africa, connectivity can be intermittent or expensive, and the typical development machine is a laptop with no discrete GPU and 8 to 16 GB of RAM.

Gemma 4 includes edge-oriented variants (E2B and E4B) that Google describes as designed to run offline on laptops and phones, with "effective" parameter counts enabled by Per-Layer Embeddings, a 128K-token context window, and native function calling [Google DeepMind, Gemma 4 model page; Gemma docs]. Ollama distributes them locally (e.g., the `gemma4:e2b` tag) [Ollama, gemma4 library page].

But agent scaffolds were mostly designed for much larger models. When we ported a standard ReAct loop [Yao et al., 2022] to a 2B-class model on a CPU-only laptop, the scaffold itself became the main source of failure. The model could often write correct code but could not reliably follow the protocol around it.

This paper reports on GemmAI, an offline developer agent built by an independent student developer. Our central argument is:

> **For small local models, reliability should be moved out of the model and into the harness.** Prefer one-shot direct actions in a format the model already knows (Markdown code blocks) over multi-step reasoning protocols. Then wrap every action in deterministic verification, recovery, and reversible version control.

Contributions.

1. A qualitative taxonomy of failure modes we observed when running a verbose ReAct agent on small local models (Section 3).
2. The GemmAI direct-action architecture and its safeguards, each mapped to the failure mode it addresses (Sections 4 and 5).
3. A deployment path for offline use on consumer machines (Section 6).
4. A proposed, reproducible evaluation protocol for small-model offline coding agents, which we have not yet run (Section 7).

Scope and honesty statement. This is a system-design and experience paper. **It reports no quantitative success rates, latencies, or benchmark scores.** Described behaviors come from qualitative observation during development, plus one verified end-to-end example; Section 7 states how to turn them into measured claims.

2. Background and Related Work

ReAct. Yao et al. (2022) proposed interleaving free-form reasoning traces ("Thought") with actions ("Action") and environment feedback ("Observation"). It became the default template for many tool-using agents.

SWE-bench. Jimenez et al. (2023) introduced SWE-bench, which evaluates whether language models can resolve real GitHub issues by producing patches that pass repository tests. Its subset SWE-bench Lite underlies our proposed protocol.

SWE-agent. Yang et al. (2024) showed that the design of the *agent-computer interface* (the commands, feedback format, and guardrails available to the model) strongly affects agent performance, separately from the underlying model. We ask what that interface should be for a small model on a CPU.

Parameter-efficient fine-tuning. QLoRA (Dettmers et al., 2023) enables fine-tuning of quantized models with low memory use.

Gemma models. The Gemma family of open models from Google DeepMind (Gemma Team, 2024, technical reports) includes small checkpoints suited to local inference. Gemma 4 adds edge variants (E2B/E4B) aimed at on-device use [Google DeepMind, Gemma 4 page].

Terminal coding assistants. Tools such as Gemini CLI popularized a direct style that proposes concrete file edits and commands. GemmAI adopts this style for fully offline use with small models.

3. Failure Modes of Verbose ReAct on Small Local Models

The first GemmAI prototype used a standard ReAct prompt. It contained a system description, a list of tools (list directory, read file, write file, run command), detailed few-shot example trajectories, and a required `Thought/Action/Action Input/Observation` format. We ran it with `gemma2:2b` via Ollama on a CPU-only machine. Three recurring failure modes made the agent unusable in practice. We describe them qualitatively; we did not measure their frequencies.

F1: Parroting prompt examples. The model often reproduced the few-shot demonstrations instead of solving the user's task. For example, it emitted file names, commands, or code taken from the example trajectories. A small model may treat demonstrations as content to copy rather than as format guidance; longer, more concrete examples appeared to make this worse.

F2: Infinite exploration loops on empty directories. When the working directory was empty (the common case of "create a new project"), the model repeatedly called the directory-listing tool, received an empty observation, and listed again. It never moved to writing a file. ReAct's "look before you act" became a loop that consumed the step budget.

F3: Parsing crashes from nested quotes. Tool inputs that contained code, especially Python strings with quotes inside a JSON- or quote-delimited action argument, were frequently malformed. A strict parser then failed and the turn was lost, even when the code itself was correct.

Common cause. All three failures come from asking a small model to do *two* hard things at once: solve the task, and conform to an unfamiliar, multi-turn, strictly formatted protocol. We therefore removed protocol burden from the model.

4. The GemmAI Direct-Action Architecture

GemmAI is a Python command-line application that talks to a local Ollama server. The user selects a model at runtime and can hot-switch with `/model` (e.g., between `gemma2:2b` and `gemma4`) without restarting the session. The redesigned core has five components.

4.1 Labeled Markdown file blocks

Instead of tool calls, the model is asked to output files directly as fenced Markdown code blocks whose info string carries the target path:

MARKDOWN FILE BLOCK

````
```fichier:hello.py
print("salut je suis GemmAI")
```
````

(`fichier` is French for "file"; the label is configurable.) Markdown code blocks are ubiquitous in training data, so even small models produce them reliably. Fenced code needs no escaping, removing F3 at its source, and one response can write a multi-file project without a tool loop. Shell commands to run are emitted the same way in `bash` blocks.

4.2 Automatic project context injection

Rather than letting the model explore the filesystem through tool calls, the harness builds a compact project summary deterministically and injects it into the prompt: the directory tree, plus file contents where size allows. If the directory is empty, the prompt says so explicitly. This removes the need for exploratory actions and so removes the empty-directory loop (F2) by construction. Gemma 4's 128K window [Gemma docs] allows larger projects, but on CPU prompt length drives processing time, so context is kept compact.

4.3 Fallback contextual-matching parser

Small models do not always follow the labeled-block format exactly. They may omit the path label, put the path in a sentence before the block, or use a plain language tag such as `python`. GemmAI's parser therefore works in two stages:

(1) a strict parser for `fichier:path` blocks;

(2) a fallback that matches unlabeled code blocks with nearby context, such as a file name mentioned in the surrounding text, a file name in the user's request, or a file already in the project with a matching extension.

The fallback turns many near-misses into usable actions instead of hard failures.

4.4 Intent filtering: chat mode vs. developer mode

Greetings and conceptual questions should not trigger file writes. A lightweight intent filter classifies each request as *chat* or *developer*. In chat mode the model answers conversationally and no file or command actions are executed. In developer mode the file-block protocol and the safeguards below are active.

4.5 Streaming output

Responses are streamed token by token to the terminal. On CPU-only inference this lets the user see progress immediately and interrupt a response going wrong.

4.6 Mapping to failure modes

FAILURE MODE	PRIMARY MITIGATION IN GEMMAI
F1 Parroting examples	Short, example-light prompt; output format (Markdown blocks) that needs no demonstration trajectories
F2 Exploration loops	Deterministic context injection; no exploratory tool loop; explicit "directory is empty" signal
F3 Nested-quote crashes	Code carried in fenced blocks (no escaping); fallback contextual parser

5. Safeguards: Moving Reliability into the Harness

A direct-action agent writes files immediately, so mistakes must be cheap to detect and to undo. GemmAI wraps every developer-mode action in the following deterministic mechanisms.

5.1 Self-healing execution loop. After writing code, GemmAI can execute it (e.g., `python3 file.py` or a requested bash command). If execution fails, the harness captures the traceback or shell error and re-prompts the model with the failing file, the exact error, and an instruction to fix it. The model rewrites the file and the harness re-runs it. The loop repeats up to a bounded number of attempts. Execution feedback, central to SWE-bench and SWE-agent, is thus used in a tight local loop needing no network.

5.2 Automatic git checkpoint and `/undo`. Before every write operation, GemmAI creates a git checkpoint (initializing a repository if needed). The `/undo` command restores the previous state. Every action is reversible, which matters when a small model overwrites working code with a worse version.

5.3 Colored unified diff. Every modification of an existing file is shown to the user as a colored unified diff, so changes stay transparent and reviewable even when the model rewrites whole files.

5.4 File-mention context injection. When the user mentions a file by name in a request, its current content is injected into the prompt automatically. The model edits what is on disk rather than hallucinating it.

5.5 Multi-step write verification. After each write the harness checks that (a) the file exists, (b) it is non-empty, (c) re-reading it from disk returns exactly the content that was intended, and (d) for Python files, `python3 -m py_compile` succeeds. A syntax error caught at step (d) feeds directly into the self-healing loop before the code is ever run.

Design principle. Each safeguard is deterministic and model-independent, so it applies unchanged after a `/model` switch and does not rely on the model checking its own work.

5.6 Verified example

A minimal end-to-end run, verified by the author: in developer mode, GemmAI was asked to create a Python script that prints a greeting. The model produced a labeled file block, and the harness wrote the file, passed write verification, and executed it. The script printed `salut je suis GemmAI` without error. It illustrates the full pipeline (intent filter → file block → parser → checkpoint → verification → execution); it is not evidence of general capability.

6. Offline Deployment on Consumer Hardware

Target environment. CPU-only machines with 16 GB RAM or less, no cloud API, no required network access after installation.

Model serving. Models are served by Ollama. GemmAI has been exercised with `gemma2:2b` and with Gemma 4 through Ollama (`gemma4`). The E2B edge variant (`gemma4:e2b`; 2.3B effective parameters, 128K context per the Ollama model card) is the intended default for low-memory machines [Ollama library]. The `/model` command lets users trade quality for speed per task.

Installation. A one-line `install.sh` script installs dependencies and creates a global `gemmai` command, so the agent can be launched from any project directory.

Standalone packaging (in progress). To remove the Ollama dependency, we are building a single executable (PyInstaller) bundling a GGUF-quantized model run through `llama-cpp-python`, so the agent can be distributed on a USB drive or downloaded once over a slow connection.

7. Proposed Evaluation Protocol (Future Work)

No evaluation results are reported in this paper. To make our qualitative claims verifiable, we propose the following protocol, which we intend to run and release.

Research questions.

- RQ1: Does the direct-action architecture resolve more tasks than a ReAct baseline on the same small model?
- RQ2: How much does each safeguard (self-healing loop, write verification, context injection, fallback parser) contribute? (ablation)
- RQ3: How do `gemma2:2b`, `gemma4:e2b`, and `gemma4:e4b` compare under identical harnesses on CPU-only hardware?
- RQ4: How often do the failure modes F1–F3 occur under each configuration?

Task suites.

1. *SWE-bench Lite-style* repository issue resolution [Jimenez et al., 2023], possibly restricted to a subset whose repositories fit the context and memory budget of the target machine.
2. A *greenfield suite* of small "create a project from scratch" tasks (scripts, CLI tools, small web pages) with hidden unit tests. This suite reflects the empty-directory scenario in which F2 was observed.

Conditions. (a) ReAct baseline; (b) direct-action without safeguards; (c) full GemmAI; (d) leave-one-out ablations of each safeguard. The model, quantization, prompt, and maximum self-healing iterations are held fixed within each comparison.

Metrics. Task resolution rate (tests pass); number of self-healing iterations; wall-clock time per task and peak RAM on a documented CPU-only machine; parse-failure rate; frequency of each failure mode (F1–F3), labeled by a fixed rubric; `/undo` usage in user sessions.

Reproducibility. We will log hardware, OS, Ollama/model versions and tags, quantization, sampling settings, prompts, and all transcripts including failed runs.

8. Discussion, Limitations, and Roadmap

Why direct action may generalize. When model capacity is scarce, the scaffold should not compete with the task for it. Formats the model already knows well (Markdown code blocks), deterministic context gathering, and deterministic verification are cheap for the harness and expensive for a small model. We hypothesize this transfers to other small models; Section 7 would test it.

Trade-offs. Direct action gives up some of ReAct's flexibility. GemmAI does not autonomously search large repositories, and context injection is bounded by CPU prompt-processing cost. Whole-file rewrites can be wasteful for large files. Gemma 4's native function calling [Gemma docs] may reduce the formatting problems that motivated our design; comparing it against file blocks on E2B is a natural next experiment.

Limitations. (1) No quantitative evaluation yet; all failure-mode observations are qualitative. (2) Development and testing were done by a single developer on a limited set of tasks and machines. (3) The intent filter and fallback parser are heuristic and can misclassify or mismatch. (4) Git checkpoints make file changes reversible but do not sandbox side effects of executed commands.

Roadmap. The core architecture and robustness safeguards are implemented. Standalone packaging (GGUF + llama-cpp-python + PyInstaller) is in progress. Planned next steps: (i) run the Section 7 protocol, with SWE-bench-style benchmarking; (ii) QLoRA fine-tuning [Dettmers et al., 2023] of a small Gemma model on file-block-formatted coding trajectories and self-healing repair traces, to make the protocol native to the model; (iii) sandboxed execution.

9. Conclusion

GemmAI shows that a usable offline coding agent can be built around small Gemma models on CPU-only consumer hardware, provided the scaffold is designed for the model's limits. Verbose ReAct prompting led small models to parrot examples, loop on empty directories, and break on nested quotes. A direct-action design addresses each of these failures with labeled Markdown file blocks, injected context, tolerant parsing, and intent filtering. Self-healing execution, git checkpoints with undo, diffs, and write verification put reliability in deterministic code. For developers on modest hardware and unreliable connectivity, in Benin and elsewhere, this is a practical, cloud-free route to agentic software engineering; our next step is to measure it rigorously.

References

- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models.* arXiv:2210.03629 (ICLR 2023).
- Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., & Narasimhan, K. (2023). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* arXiv:2310.06770 (ICLR 2024).
- Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., & Press, O. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering.* arXiv:2405.15793 (NeurIPS 2024).
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.* arXiv:2305.14314 (NeurIPS 2023).
- Gemma Team, Google DeepMind (2024). *Gemma: Open Models Based on Gemini Research and Technology.* arXiv:2403.08295; and *Gemma 2: Improving Open Language Models at a Practical Size.* arXiv:2408.00118.
- Google DeepMind. *Gemma 4* model page. https://deepmind.google/models/gemma/gemma-4/
- Google AI for Developers. *Gemma documentation.* https://ai.google.dev/gemma/docs/core
- Ollama. *gemma4* model library. https://ollama.com/library/gemma4